# MIT 6.006

13 October 2026 12:35

Lecture 1: Algorithms & Computations

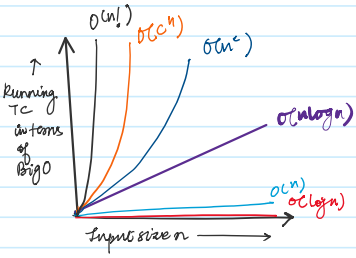
Goal → 1) solve computational problems
2) prove correctness
3) argue efficiency
4) communication of problems

Correctness: Use Induction to prove correctness in recursion questions
Base Case: $n=1$ or $n=0$
Inductive hypothesis: assume the statement is true for some arbitrary natural number
Inductive step: using the hypothesis assumption, prove it's true for $k+1$

Problem: possible inputs & outputs
↳ Computational problem: binary relation b/w set of inputs & outputs

Efficiency
By counting fundamental operation → also referred to as Frequency count method.
Use Asymptotic analysis that helps measure ops
→ Don't measure time but operation
→ expect performance to depend on size of input
Types: 1) Big O (upper bound)
2) Big Omega $\Omega$ (lower bound)
3) Theta ($\Theta$) - (both-average)

* Refer to Abdul Bari notes on TC + SC




O(1) O(logn) O(n) O(nlogn) O(n^2) O(n^ck) O(2^n)

How to learn to map with computer's memory?

RAM (Random access memory) – we assume computer can access any address in memory in constant time
memory is Byte addressable
Memory is conceptually → an arrangement of addresses

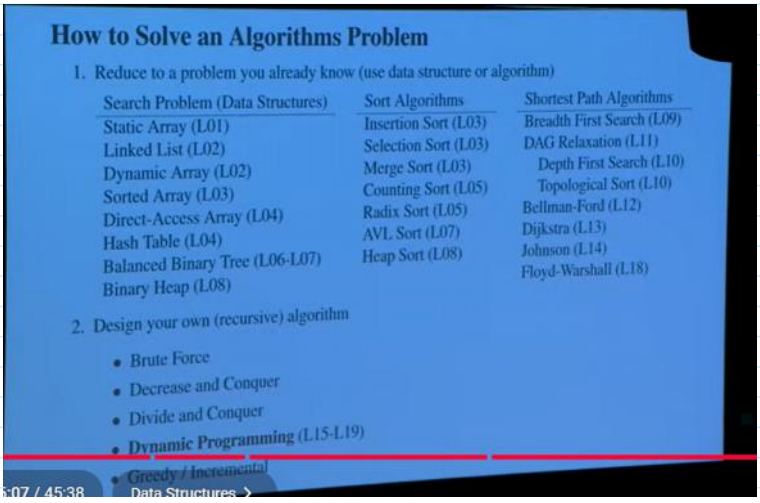



↳ 8 bits = 1 byte
1 bit is 0 or 1

What about 64 bits in computers?
That means CPU has registers that can hold 64 bits = 8 bytes × 8 bits




$= 2^n$ distinct addresses $= 2^{64}$




64 bit computer can have gigabytes of RAM with billion bytes
64 describe the width of certain processors operations, registers and address-related capabilities

↳ RAM capacity → storage available to hold data
Memory address → a bit within the addressable space
64 bit register → inside CPU for operations
64 bit word → natural data unit → CPU's natural processing width
Word size → unit of data that processor can handle in a single operation
= native data size of the architecture = 64 bits

Lecture 2: Data structures & Dynamic arrays

Three fundamental concepts:
1) Abstract data type (ADT)
what operations are available? Behaviors of data type
The contraction stack:
push(x), pop(), peek(), LIFO order
→ standford course focus on them

2) Data structure (Representation)
How is data organized & stored to support those operations?
Eg: stack can be implemented using dynamic array or a linked list
→ linear
→ non linear
→ how elements reorganized

3) Algorithm
what sequence of computational steps support those operations?

2 main interfaces:
1) Sequence
2) Set

+ 2 main data structure approaches
1) Arrays
2) Pointers.